

Day-65-CSV Files in Python

1. Day Number

Day 65

2. Topic Name

Reading and Writing CSV Data

Today you will learn how Python can work with data stored in **rows and columns** using the built-in `csv` module.

3. Connection

You already learned how to work with **normal text files**.

A text file might contain:

```
Apple costs 50
Banana costs 30
Mango costs 80
```

Today, you will work with more structured data:

```
product,price
Apple,50
Banana,30
Mango,80
```

This structure is called **CSV**.

CSV files are very common for storing simple data such as:

- products and prices
 - students and marks
 - employees and salaries
 - names and phone numbers
-

4. Important Topics

For Day 65, focus on these concepts:

- CSV
- rows
- columns
- header
- Python's `csv` module

- reading CSV files
- writing CSV files
- `csv.reader()`
- `csv.writer()`

You do **not** need Pandas yet.

5. Foundational Notes

What is CSV?

CSV stands for:

Comma-Separated Values

A CSV file is usually a normal text file where values are separated using commas.

Example:

```
product,price
Keyboard,1200
Mouse,500
Monitor,9000
```

The file contains **rows** and **columns**.

Here:

```
product    price
Keyboard   1200
Mouse      500
Monitor    9000
```

There are two columns:

```
product
price
```

And three data rows.

What is a row?

A **row** represents one record.

For example:

```
Keyboard,1200
```

This is one product record.

Another row:

```
Mouse,500
```

What is a column?

A **column** represents one type of information.

For example:

```
product,price
```

product is one column.

price is another column.

What is a header?

The first row often describes what each column means.

Example:

```
product,price
```

This is the **header row**.

The remaining rows contain actual data:

```
Keyboard,1200  
Mouse,500  
Monitor,9000
```

A CSV file does not technically require a header, but headers make the data much easier to understand.

Python's csv module

Python already includes a module called:

```
csv
```

You import it using:

```
import csv
```

You do not need to install anything with pip.

The csv module provides tools for safely reading and writing CSV data.

Two beginner-friendly tools are:

```
csv.reader()
```

and:

```
csv.writer()
```

Think of them like this:

```
csv.reader()  
  ↓  
CSV file → Python rows
```

```
csv.writer()  
  ↓  
Python data → CSV file
```

6. CSV vs Normal Text File

Both CSV files and normal text files store text.

The main difference is **structure**.

A normal text file might contain:

```
Keyboard costs 1200 rupees  
Mouse costs 500 rupees  
Monitor costs 9000 rupees
```

Python would have to figure out where the product name ends and where the price begins.

A CSV file makes the structure clearer:

```
product,price  
Keyboard,1200  
Mouse,500  
Monitor,9000
```

Now Python can treat each line as a row and each comma-separated value as a column.

So you can think of it as:

```
Normal text file  
↓
```

Mostly designed for sentences/text

CSV file

↓

Designed for structured rows and columns

7. Easy Example

Suppose you have a file named:

```
students.csv
```

Containing:

```
name,score
Asha,85
Ravi,92
Neha,78
```

Conceptually, Python can read it like:

```
Row 1 → ["name", "score"]
Row 2 → ["Asha", "85"]
Row 3 → ["Ravi", "92"]
Row 4 → ["Neha", "78"]
```

Each row can be processed one at a time.

A small reading pattern looks like:

```
import csv

with open("students.csv", "r") as file:
    reader = csv.reader(file)

    for row in reader:
        print(row)
```

Possible output:

```
['name', 'score']
['Asha', '85']
['Ravi', '92']
['Neha', '78']
```

Notice something important:

CSV values are normally read as **strings**.

For example:

```
"92"
```

is initially text, not the integer:

```
92
```

You can convert values later when necessary.

8. Problem Statement

Create a small CSV file that contains:

```
product name  
price
```

For example, your file could contain products such as:

```
Keyboard  
Mouse  
Monitor
```

Your program should:

1. Create or use a CSV file.
2. Store a header containing product and price.
3. Store several products.
4. Read the CSV file.
5. Skip or handle the header appropriately.
6. Print each product and its price.

Your goal is to produce output similar to:

```
Product: Keyboard, Price: 1200  
Product: Mouse, Price: 500  
Product: Monitor, Price: 9000
```

Do not use Pandas.

9. Concepts Used

You will practice:

- importing modules

- csv module
- file handling
- with open(...)
- reading files
- writing files
- loops
- lists
- indexing
- rows and columns
- checking missing data
- functions

You will also reuse concepts from your earlier Python lessons.

10. Thought Process

Before writing code, think about the data.

Your CSV needs two columns:

```
product  
price
```

So each normal data row should contain two values:

```
Keyboard | 1200  
Mouse    | 500  
Monitor  | 9000
```

In CSV form:

```
Keyboard,1200  
Mouse,500  
Monitor,9000
```

When Python reads a row such as:

```
Keyboard,1200
```

you can imagine it becoming:

```
["Keyboard", "1200"]
```

Therefore:

```
row[0]
```

represents the product name.

And:

```
row[1]
```

represents the price.

The overall thinking process is:

```
Open file
  ↓
Create CSV reader
  ↓
Read rows
  ↓
Get product and price
  ↓
Print them
```

11. Beginner-Friendly Pseudocode

Do not write the complete Python program yet.

Start with this pseudocode:

```
import the csv module

create a function for writing products

    open products.csv in write mode

    create a CSV writer

    write the header:
        product
        price

    write a few product rows

create another function for reading products

    open products.csv in read mode

    create a CSV reader
```

```
handle the header
```

```
loop through each remaining row
```

```
    check whether the row contains enough values
```

```
    get product name
```

```
    get price
```

```
    print product and price
```

```
call the writing function
```

```
call the reading function
```

This keeps the program separated into small responsibilities.

12. Suggested Solving Approach: Functional Approach

Instead of putting everything together, divide the program into functions.

Conceptually:

```
write_products()
```

responsibility:

```
Create/write the CSV data
```

And:

```
read_products()
```

responsibility:

```
Read and display the CSV data
```

Your program structure could therefore look like:

```
imports
```

```
write_products function
```

```
read_products function
```

```
call write_products  
  
call read_products
```

This is useful because each function has **one main job**.

For example:

```
write_products()  
  ↓  
products.csv  
  ↓  
read_products()  
  ↓  
terminal output
```

This is a simple introduction to organizing programs into reusable functions.

13. Easy Edge Cases

Edge Case 1: Empty CSV File

Imagine:

```
products.csv
```

exists but contains nothing.

Your reader should not crash just because there are no products.

Conceptually:

```
Open file  
↓  
No rows found  
↓  
Print something like:  
"No products found."
```

You do not have to build complicated validation yet.

Edge Case 2: Missing Value

Imagine this CSV:

```
product,price  
Keyboard,1200
```

```
Mouse,  
Monitor,9000
```

The Mouse row is missing its price.

When Python reads it, it may look approximately like:

```
["Mouse", ""]
```

Before using the price, you could check:

```
Is the price empty?
```

If yes, you might print:

```
Mouse has no price
```

or skip that row.

For Day 65, simple handling is enough.

Another Possible Problem: Missing Column

Consider:

```
product,price  
Keyboard,1200  
Mouse  
Monitor,9000
```

The Mouse row contains only one value.

Trying to access:

```
row[1]
```

could cause a problem because that second item does not exist.

A beginner-friendly check is conceptually:

```
if row contains at least two values:  
    process the row  
else:  
    handle the incomplete row
```

14. Expected File and Output

Your project might look like:

```
day65/  
├── main.py  
└── products.csv
```

Your `products.csv` could look like:

```
product,price  
Keyboard,1200  
Mouse,500  
Monitor,9000
```

After running your program, your expected output could be:

```
Product: Keyboard, Price: 1200  
Product: Mouse, Price: 500  
Product: Monitor, Price: 9000
```

The exact products and prices can be different.

The important goal is:

```
CSV file  
  ↓  
Python reads each row  
  ↓  
Python extracts columns  
  ↓  
Python prints meaningful information
```

15. Hint Only

Start with:

```
import csv
```

For writing, investigate:

```
csv.writer()
```

For reading, investigate:

```
csv.reader()
```

Remember that one row such as:

```
Keyboard,1200
```

will roughly behave like:

```
["Keyboard", "1200"]
```

So think about how you could use:

```
row[0]  
row[1]
```

inside a loop.

For the functional approach, think in terms of:

```
write_products()  
read_products()
```

Then ask yourself:

What single responsibility should each function have?

Your Day 65 challenge is to build the program from this structure without copying a complete solution.